

Breaking the Guardrails: Empirical Insights into Systems Effectiveness Against Jailbreak Attacks in Socio-Critical Scenarios

Silvia Parolin^{[silvia.parolin@mail.polimi.it]}, Alessandro Lodetti^{[alessandro.lodetti@mail.polimi.it]}, Matteo Camilli^{[matteo.camilli@polimi.it]}, Giovanni Ennio Quattrocchi^{[giovanni.quattrocchi@polimi.it]}, and Vincenzo Scotti^{[vincenzo.scotti@kit.edu]}

Politecnico di Milano
Piazza Leonardo da Vinci 32, 20133, Milano (MI), Italy
<https://www.polimi.it/>

Abstract. The integration of Large Language Models (LLMs) into socio-critical domains raises significant safety concerns coming from both their inputs, where malicious prompts may exploit vulnerabilities, and their outputs, which may contain harmful or policy-violating content. This study focuses particularly on jailbreak attacks, a class of adversarial inputs designed to bypass built-in safety measures and force unsafe model behavior. We present a systematic, empirical evaluation of four ready-to-use guardrail systems: Google’s Perspective API, Guardrails AI’s Detect Jailbreak, Azure’s Prompt Shields, and Meta’s Llama Guard. The study purpose is to assess their ability to detect jailbreak attacks: using various public datasets containing both benign and adversarial prompts, we measure performance via precision, recall, accuracy, specificity, and F1 score. Results show that all systems prioritize high precision over recall, effectively minimizing false positives but leaving many unsafe prompts undetected. Llama Guard consistently achieves the highest recall and identifies the largest number of unsafe prompts, while other systems’ behavior can depend on the prompting style, particularly on explicit versus obfuscated threats. Our findings highlight the absence of a universally robust solution and underline the need for hybrid, context-aware approaches that enhance detection coverage without compromising precision. This study provides practical insights for developers deploying LLMs in socio-critical applications, aiming at the design and production of more resilient and ethically aligned AI safety tools.

Keywords: Guardrail Systems · Large Language Models · Generative Artificial Intelligence · Socio-Critical Systems · Jailbreak Attacks

1 Introduction

Recent advancements in Artificial Intelligence (AI), particularly in the development of Large Language Models (LLMs) for Generative AI applications (e.g.,

ChatGPT), have significantly transformed the ways in which machines interact with humans and the world. These models, often deployed in various socio-critical domains, play now active roles in assisting users in everyday tasks, and making decisions that may carry ethical, legal, or safety implications. However, an increased risk of misuse comes with their growing integration into high-stakes environments, especially through techniques known as jailbreak attacks. These attacks are designed to bypass the safety mechanisms natively embedded in LLMs, prompting them to generate harmful, biased, or otherwise restricted content [1], [2].

To mitigate these risks, several guardrail systems have been introduced. These systems aim to safeguard both the input and output channels of LLMs by dually detecting adversarial prompts and preventing the generation of unsafe responses.[3] Guardrails operate by enforcing safety, ethical, and task-specific constraints, and they are becoming a fundamental element in the deployment of responsible AI. While many of these tools have been proposed in academic literature or released as part of commercial frameworks, a clear, empirical assessment of their practical effectiveness remains limited.

The primary objective of this study is to fill that gap by conducting a systematic, empirical evaluation of selected guardrail systems, with a specific focus on their ability to detect and handle jailbreak attacks. The paper limits its scope to publicly available tools that can be readily integrated into applications without proprietary customization, in order to provide insights that are directly applicable to developers, researchers, and practitioners. Given the heightened risks associated with socio-critical applications, a comprehensive analysis of these tools is essential to ensure their correct and responsible usage, as the consequences of unsafe model behavior in such contexts can be especially severe.

To achieve this goal, we evaluate a range of guardrail solutions, both model-based and framework-based. These tools were tested using standardized datasets and metrics to ensure comparability and reproducibility. By examining their performance across a variety of attack scenarios, we aim to identify their strengths, limitations, and suitability for real-world deployment.

The structure of this paper is as follows. Section 2 provides an overview of existing guardrail systems and defines the key concepts of jailbreaking and safety in LLMs. Section 3 details the experimental setup, including datasets, methodology, and evaluation metrics. Section 4 presents the empirical findings and discusses their implications. Finally, Section 5 concludes the paper and outlines directions for future research in the development of robust, ethically aligned AI systems.

2 Background and Related Work

2.1 Existing Guardrail Systems for LLMs

Definition 1. *Guardrail Systems for products powered by Generative AI* are frameworks, tools, or set of rules designed to constrain and regulate the behavior of Large Language Models, in the context of Generative AI, in order to ensure that their inputs and outputs have safeguards against generating high-risk

or policy-violating content as well as to protect against adversarial inputs and attempts at jailbreaking the model.[4] In other words, guardrails ensure compliance with defined safety, ethical, legal, or task-specific requirements to prevent the generation of harmful, biased, unsafe, or otherwise undesired content.

Definition 2. *Jailbreak Attack* is a type of vulnerability where adversaries craft malicious inputs to bypass safety alignments of LLMs and elicit harmful, biased, or unintended responses.[5]

Numerous guardrail systems have been developed and proposed to enhance the safety and reliability of LLMs, particularly in socio-critical contexts. For instance, they can be employed to monitor conversations between human users and LLM-based generative agents to ensure that both parties adhere to ethical standards. Some guardrails are designed to detect harmful or biased outputs generated by LLMs, while others aim to prevent manipulative user inputs such as jailbreak attempts, or other forms of misuse. A few systems are capable of addressing both concerns. The most prominent guardrails currently available can be assigned to one of two categories: model-based or framework.

Model-based Guardrail Systems These systems rely on specialized machine learning models trained to detect or classify unsafe or harmful inputs and outputs. For each model-based system we report a brief description below.

Llama Guard, developed by Meta, is a safety framework designed to mitigate harmful or toxic outputs generated by LLMs, particularly Meta’s LLaMA series. It operates as an intermediary filter between the base model’s output (e.g., LLaMA 2 7B, Mistral models) and the end-user, ensuring safer and more ethical text generation. [4] See Section 2.2 for details.

Prompt Guard is a small open-source model by Meta: it categorizes input strings into three simple categories, which are benign, injection, and jailbreak.[6]

PerspectiveAPI is an API developed by Google which uses machine learning models to score the perceived impact a comment might have on a conversation. Therefore, it can be used to analyse and pre-filter user prompts.[7] See Section 2.2 for details.

R² Guard introduces robust reasoning into LLM-based guardrails for user prompts using knowledge-enhanced logical inference. It uses first-order logical rules compiled into probabilistic graphical models to compute unsafety probabilities, then evaluates prompts across different safety categories with dedicated classifiers and outputs the probability of the prompt being unsafe. [8]

GradSafe is focused on detecting jailbreak prompts. It is available in two variants: *GradSafe-Zero* checks whether cosine similarity across some pre-defined safety-critical parameters is over the set threshold to decide if a prompt is unsafe, whereas *GradSafe-Adapt* adapts the model’s weighting of these parameters via logistic regression for improved detection.[9]

Wildflare Guardrails Employs a unified binary classification model fine-tuned on a custom dataset to detect unsafe content.[10]

Detoxify is aimed at mitigating toxic content in text data, therefore it can be used to pre-screen user prompts. It leverages pre-trained models to classify text into categories such as toxicity, severe toxicity, obscenity, threats, insults, and identity-based hate.[11]

Framework Guardrail Systems These systems are software frameworks or middleware tools that control the behavior of LLMs by applying logic, filters, and rules between the user and the LLM. As we did for the model-based systems, for each framework we report a brief description below.

NeMo Guardrails, developed by NVIDIA, is not a standalone model but a framework that allows developers to easily insert programmable guardrails between application code and adopted LLMs.[12]

Amazon Bedrock Guardrails is integrated within the Amazon Bedrock service. It can be used to safeguard Gen-AI applications that rely on LLMs within Amazon Bedrock Cloud. The content is filtered across 6 categories which are: hate, insults, sexual, violence, misconduct, and prompt attacks.[13]

Detect Jailbreak is a jailbreak prevention validator developed by Guardrails AI which can identify jailbreaking attempts in real-time, preventing the generation of harmful content. [14] It can be found among the tools presented in the Guardrails AI hub. [15] For further details, see Section 2.2

Prompt Shields is a unified API in Microsoft’s Azure AI Content Safety that detects and blocks adversarial user input attacks on large language models (LLMs), as well as third-parties attacks. [16] To maintain consistency with the other guardrails, this paper focuses only on user attacks. See Section 2.2 for additional information.

Guidance applies predefined (customizable) rules and formats to steer the LLM output to ensure that responses adhere to specified structural and safety requirements. [17]

2.2 Guardrail Systems analyzed in this paper

Among the various guardrail systems discussed in the literature, this study focuses specifically on tools that are readily available and can be applied as-is, without requiring significant customization or proprietary integration. The rationale for this choice is to assess practical, out-of-the-box solutions that can be immediately adopted in real-world applications. Below can be found how the selected tools operate.

Llama Guard can be used to classify content in both LLM inputs (prompt classification) and in LLM responses (response classification). The output of the

guardrail indicates whether a given prompt or response is safe or unsafe, and if unsafe, it also lists the content categories violated.[18] The model is trained to predict safety labels based on the MLCommons taxonomy[19] of 13 hazards, as well as an additional category for Code Interpreter Abuse.

PerspectiveAPI is hosted on Google Cloud. The request to the API should include:

The text to be analyzed. The original purpose of Perspective was to analyze the degree of toxicity of any kind of comment left on the Internet. For the sake of this research, we test Perspective API to assess its usability as a guardrail system for user prompts.

The list of attributes one wants to receive scores for. The API predicts on a scale from 0 to 1 the perceived impact a comment or prompt may have on a conversation by evaluating that comment or prompt across a range of emotional concepts. In other words, it indicates the probability that a reader will perceive the comment provided in the request as containing the given attribute.[7] The main category is *Toxicity*.

Detect Jailbreak On their hub[20], Guardrails AI provide many software components that are used to validate an aspect of an LLM workflow. In this paper, we are focusing on Detect Jailbreak[15], which is an API built to detect and ward off attempts to jailbreak language models. The API receives the prompt to be analyzed and outputs a structured response which contains a boolean field to indicate if the validation has been passed (safe prompt) or not.

Prompt Shields is a specialized filter for adversarial inputs that try to bypass the safety protocols already integrated in a Generative AI application. Prompt Shields can detect two types of input attacks: user prompt attacks and document attacks. The first one is the attempt of the user to alter the intended LLM behavior through ad hoc prompts that aim to force the LLM to ignore constraints, policies, and ethical norms. The second one refers to the practice of making the LLM misinterpret third-party content (e.g., documents) to gain unauthorized access or control.

2.3 Related Work

Some experiments on the behavior and performance of guardrail systems have already been published. For instance, within the presentation paper of Llama Guard [4] the authors focus not only on implementation but also on how Llama Guard performs in a general scenario when queried with simple toxic prompts, not proper adversarial prompts created ad-hoc to perform an attack against the LLM. However, they only tested the guardrail in simple scenarios with a very limited number of adversarial prompts, whereas considering critical adversarial

prompts, that try to break the LLM safety in a more silent way, is the core of our paper.

Moreover, [2] and [3] describe which problems afflict LLMs, such as adversarial prompting. However, even though those two papers offer a great overview about LLM, guardrails, and which types of adversarial prompt an LLM can encounter, they do not investigate any particular system, and consequently they do not offer any statistic about the available guardrails.

In terms of Perspective API[7], the tool itself was not built to specifically face jailbreak attacks, but it has already been evaluated as a guardrail system [9]. This evaluation was made only on two datasets which contain very few real adversarial prompts. The very same paper also tests Azure Prompt Shields but on the same datasets as before. Moreover, as it was for Llama Guard, Perspective API was only tested on simple toxic prompt, hence the paper does not investigate on what happens if instead of an explicit toxic prompt a proper adversarial prompt is passed as an input.

Consequently, this work aims to fill the gap left by these studies and attempts to discover which is the guardrail that better copes against adversarial prompting. Moreover, by offering an analytic approach we also aim to extract some information about the effectiveness of Detect Jailbreak by Guardrail AI, because as far as we know there are no papers that analyzed those aspects on public datasets.

3 Experiments

3.1 Datasets

All datasets employed in our experiments contain both *safe* and *unsafe* prompts. The unsafe prompts include jailbreak attempts and other forms of policy-violating inputs, while the safe prompts represent benign user requests. Importantly, the proportion of safe and unsafe prompts is not constant across datasets. This intentional variability allows us to evaluate the guardrail’s performance under diverse conditions, including scenarios where malicious inputs are rare and others where they are prevalent. Such heterogeneity provides a more comprehensive and stress-tested assessment of detection capabilities.

3.2 Methodology

The methodology used within this paper explores four guardrail systems. They can be split into two groups for a deeper understanding of how they work, but the full set of experiments should be seen as a whole.

- The first group comprehends guardrails accessible via free API calls, subject only to service-imposed rate limits (e.g., a maximum number of requests per minute or per month). This category included Google’s *Perspective API*, Azure’s *Prompt Shield*, and *Detect Jailbreak* from Guardrails AI. Each of these guardrail system was tasked with assessing prompt safety, and then

Name	Source	#Unsafe Prompts	#Safe Prompts
Wildjailbreak	HuggingFace	2,000	210
Toxic Chat 1123	HuggingFace	113	4,969
JBB-Behaviors	HuggingFace	100	100
Wildguard Mix	HuggingFace	810	915
XSTest	HuggingFace	200	250
In the wild jailbreak prompts*	HuggingFace	1,405	1,405

*For the last dataset, only when dealing with the Llama Guard experiment, a sample was taken of 50 safe, and 86 unsafe examples due to present hardware limitations.

Table 1: Dataset structure.

the resulting predictions were compared with the ground-truth labels to calculate the metrics explained in Section 3.3.

As mentioned in Section 2.2, PerspectiveAPI evaluates user-specified categories for each prompt. This work considers the following categories: *Toxicity*, *Severe_toxicity*, *Threat*, *Insult*, *Obscene*, *Identity_attack*, *Sexually_explicit*.

In Section 2.2, it is stated that Prompt Shields Prompt Shields can detect two types of input attacks: user prompt attacks and third-party attacks. Given this paper’s purpose, the evaluation will only focus on the first type of detection, which comprises four categories of attack: *Attempt to change system rules*, *Embedding a conversation mockup to confuse the model*, *Role-Play*, *Encoding Attacks*. [16]

Note: The tool does not allow to retrieve which is the type of attack detected, but we can state that all four categories can fall into the jailbreaking case.

- The second group is composed only by *Llama Guard*, specifically Llama Guard 3-8B version, a Llama 3.1 model [21] that uses eight billion parameters fine-tuned for content safety classification. In this setting, each prompt was first used to generate a response from the underlying language model (in this case, Mistral 7B). The safety evaluation of *Llama Guard* was then performed on the full conversation, consisting of both the input prompt and the generated response. By comparing the resulting performance metrics across the two runs, it was assessed the impact of the guardrail on detection and filtering capabilities.

Python code can be found on [this](#) Github Repository.

3.3 Evaluation Metrics

To assess the performance of each guardrail system against jailbreak attacks, we report four complementary metrics: *Number of Detected Cases*, *Precision*, *Recall*, *Accuracy*, *Specificity*, and *F1 Score*.

Number of Detected Cases: The absolute count of jailbreak attempts flagged by the guardrail as malicious or policy-violating.

It provides a direct measure of detection volume. While a high number may indicate broad coverage, it can be misleading without quality measures such as false positive rates.

Precision Formula:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

This formula computes the proportion of detected jailbreaks that are actual jailbreak attempts[22].

High precision indicates that the guardrail rarely blocks benign requests, which is crucial for user trust and system usability. Excessive false alarms degrade user experience and may discourage adoption of the guardrail.

Recall Formula:

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

This metric indicates the proportion of actual jailbreak attempts that are successfully detected[22].

High recall ensures most jailbreak attempts are intercepted before unsafe model behavior occurs. This is critical for safety guarantees, especially in high-risk domains.

F1 Score Formula:

$$\text{F1} = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

This score is the harmonic mean of precision and recall.[22]

F1 balances detection accuracy and coverage, penalizing systems that optimize one at the expense of the other. It is particularly useful when both false positives and false negatives carry significant costs.

Accuracy Formula:

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Predictions}}$$

This metric reflects the overall correctness of the guardrail’s predictions[22].

High accuracy suggests that the system correctly identifies both jailbreak and benign cases. However, it can be misleading in imbalanced datasets, where one class dominates, as high accuracy may mask poor performance on the minority class.

Specificity Formula:

$$\text{Specificity} = \frac{\text{True Negatives}}{\text{True Negatives} + \text{False Positives}}$$

This metric measures the guardrail’s ability to correctly allow benign inputs[22]. High specificity ensures that legitimate queries are not mistakenly blocked, preserving normal user interactions. It complements recall by capturing the system’s tolerance to false positives, which is vital for maintaining usability.

4 Results and Discussion

As explained earlier in this paper, the core methodology involves to compute precision, recall, accuracy, specificity, and F1-score by using datasets containing both adversarial and non-adversarial prompts, which are all processed by various guardrails. Tables 2 through 5 present the results achieved for our tests, which can be found below.

Dataset	Unsafe prompts	True Positive	Detected cases	Precision	Recall	F1 score	Accuracy	Specificity
Wildjailbreak	2000	30	30	1.0	0.015	0.029	0.1086	1
Toxic-chat 1123	113	32	410	0.078	0.283	0.123	0.910	0.924
JailbreakBench	100	11	15	0.733	0.110	0.191	0.535	0.960
Wildguardmix	810	6	27	0.207	0.007	0.014	0.521	0.976
XStest	200	69	90	0.767	0.345	0.476	0.662	0.916
Jailbreak_llms	1405	361	460	0.785	0.257	0.338	0.593	0.930

Table 2: Results for PerspectiveAPI.

Dataset	Unsafe prompts	True Positive	Detected cases	Precision	Recall	F1 score	Accuracy	Specificity
Wildjailbreak	2000	206	208	0.888	0.103	0.185	0.176	0.876
Toxic-chat 1123	113	71	202	0.351	0.628	0.451	0.966	0.974
JailbreakBench	100	1	2	0.5	0.010	0.02	0.500	0.990
Wildguardmix	810	100	105	0.943	0.123	0.218	0.585	0.993
XStest	200	0	0	0	0	0	0	0
Jailbreak_llms	1405	398	530	0.750	0.283	0.785	0.594	0.906

Table 3: Results for Detect Jailbreak.

Dataset	Unsafe prompts	True Positive	Detected cases	Precision	Recall	F1 score	Accuracy	Specificity
Wildjailbreak	2000	252	263	0.958	0.126	0.122	0.204	0.947
Toxic-chat 1123	113	74	110	0.673	0.655	0.664	0.985	0.993
JailbreakBench	100	0	0	0	0	0	0	0
Wildguardmix	810	103	103	1	0.128	0.228	0.590	1
XStest	200	0	0	0	0	0	0	0
Jailbreak_llms	1405	1034	1432	0.722	0.736	0.729	0.726	0.717

Table 4: Results for Prompt Shields.

Dataset	Unsafe prompts	True Positive	Detected cases	Precision	Recall	F1 score	Accuracy	Specificity
Wildjailbreak	2000	311	313	0.994	0.156	0.269	0.235	0.990
Toxic-chat 1123	113	21	146	0.146	0.189	0.165	0.957	0.975
JailbreakBench	100	64	68	0.941	0.640	0.762	0.800	0.960
Wildguardmix	810	142	150	0.951	0.176	0.297	0.609	0.992
XStest	200	126	138	0.913	0.630	0.7456	0.809	0.952
Jailbreak_llms	86	8	8	1	0.152	0.263	0.688	1

Table 5: Results for Llama Guard.

Result charts are shown in Figure 1, 2, 3, 4, 7.

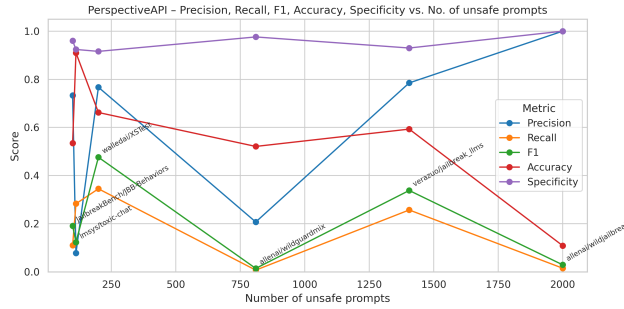


Fig. 1: Metrics results for PerspectiveAPI.

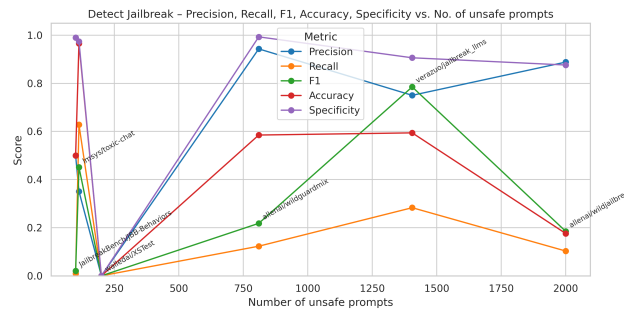


Fig. 2: Metrics results for Detect Jailbreak.

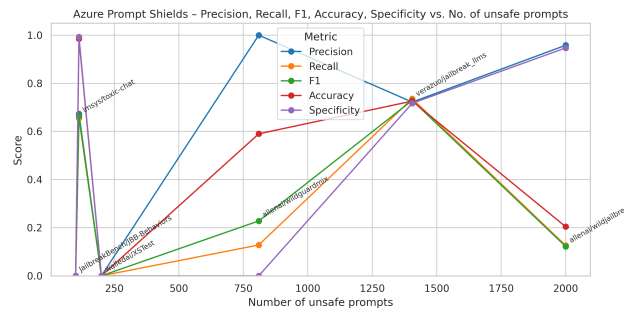


Fig. 3: Metrics results for Prompt Shields.

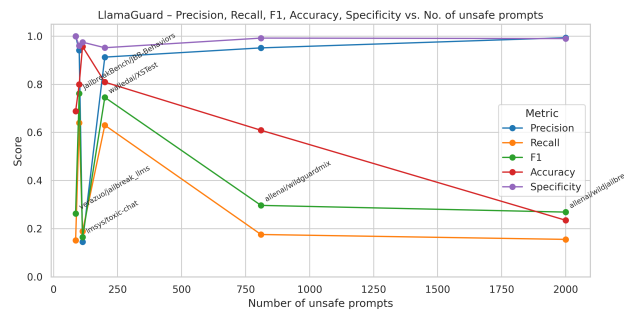


Fig. 4: Metrics results for Llama Guard.

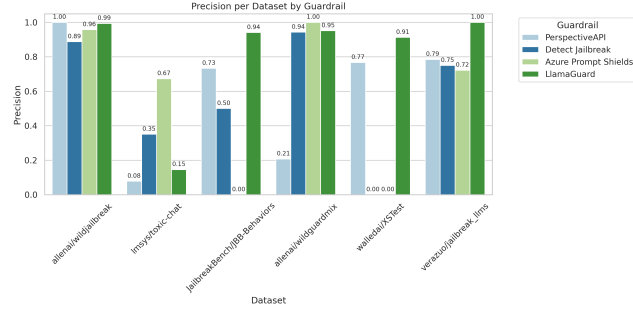


Fig. 5: Comparison of Precision values achieved by the guardrails across all datasets.

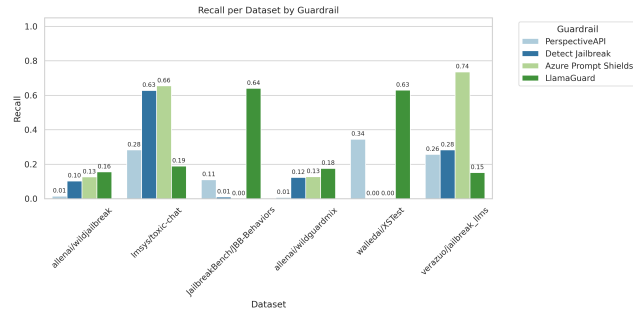


Fig. 6: Comparison of Recall values achieved by the guardrails across all datasets.

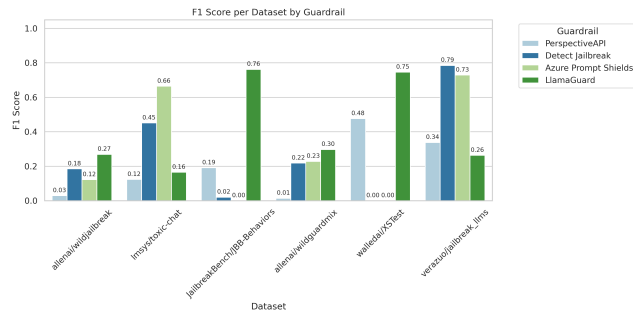


Fig. 7: Comparison of F1 values achieved by the guardrails across all datasets.

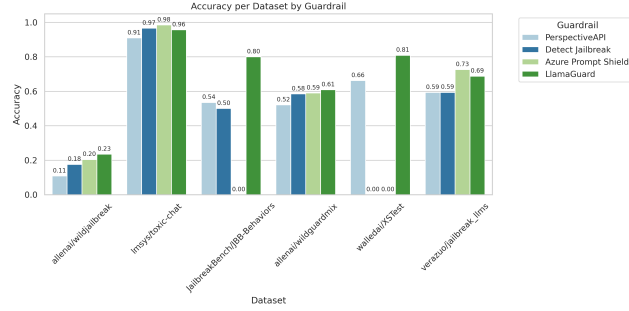


Fig. 8: Comparison of Accuracy values achieved by the guardrails across all datasets.

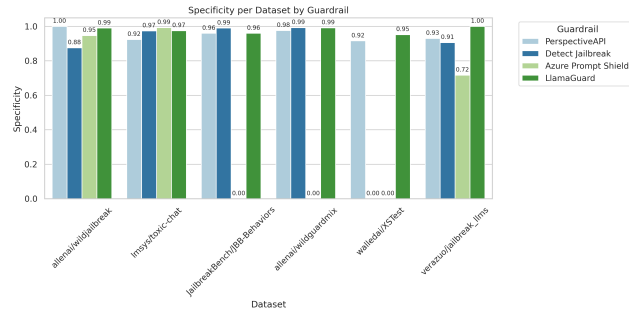


Fig. 9: Comparison of Specificity values achieved by the guardrails across all datasets.

Across all four systems (Perspective API, Detect Jailbreak, Azure Prompt Shields, and Llama Guard) the observed performance trends indicate that precision is generally high for many datasets, in some cases approaching 1.0, meaning not many false positives are detected. In general, Azure Prompt Shields and Detect Jailbreak exhibit similar precision patterns, peaking on datasets of intermediate size, whereas Perspective API displays greater variability. Llama Guard, apart from the aforementioned exception, shows optimal precision values (between 0.9 and 1). However, this behavior is inconsistent, with substantial variation across datasets. In the subsection below (4.1) we report the main exceptions.

On the contrary, recall values are consistently low for the majority of datasets, indicating that a considerable proportion of unsafe prompts remain undetected. Azure Prompt Shields achieves its highest recall (~ 0.7) on a smaller dataset, with performance declining as dataset size increases. Detect Jailbreak and Perspective API display similar recall profiles, characterized by low overall values

with only modest improvements on smaller datasets.

As expected from the low recall, F1-scores are generally modest, reflecting the difficulty in achieving a balanced trade-off between precision and recall. For Detect Jailbreak, Azure’s Prompt Shields, and Llama Guard, F1 performance tends to improve for datasets in the mid-range of unsafe prompt counts, where recall increases without a substantial loss in precision. Perspective API exhibits more irregular F1 behavior, with pronounced drops in cases where either precision or recall declines sharply.

Accuracy values are generally moderate across most datasets and guardrails, with some of them even close to 0 or 0.1 (e.g., Wildjailbreak with PerspectiveAPI). This metric has a peak on the smallest dataset (>0.9) but the general correctness of the analyzed guardrails in classifying prompts as safe or unsafe is limited, especially in datasets with large numbers of unsafe prompts or imbalanced classes.

On the other hand, specificity values are consistently high, often close to or equal to 1.0, across all datasets and guardrails. For example, Wildjailbreak with PerspectiveAPI shows perfect specificity (1.0), and many other datasets have specificity above 0.9.

This combination suggests that the guardrails are very good at correctly identifying safe prompts (true negatives), but less effective at correctly identifying unsafe prompts (true positives). In other words, guardrails tend to avoid false alarms (few false positives), reflected in high specificity, but at the cost of missing many unsafe prompts, lowering recall and accuracy. Overall, all four systems appear to adopt a conservative classification strategy, favoring high precision at the cost of reduced recall. The highest F1-scores are achieved when a better balance between the two metrics is maintained, which is not necessarily associated with the maximum precision values. Therefore, improving guardrails should focus on increasing recall (detecting actual true positives) without severely impacting precision and specificity to improve overall F1 and accuracy. When dealing with implementation and deployment of these systems, the results of this series of experiments display the need for balanced trade-offs depending on use case priorities (e.g., safety vs. minimizing false alarms).

Each metric result organized by dataset can be visualized in Figures 5, 6, 7, 8, 9.

4.1 Exceptions

The Toxic Chat Problem Evaluations conducted on the *Toxic Chat* dataset present slightly higher recall than the general trend, but definitely lower precision. These metrics reveal that a greater number of unsafe prompts are detected, but more false positives are reported too. The reason behind this could be the linguistic complexity, featuring examples that intentionally obscure or embed unsafe content in apparently benign language, as well as the deep context dependence. This dataset shows an intrinsic detection difficulty, as indicated by prior performance of established moderation tools on similar experiments[4].

The Explicitness Problem When analysing two distinct datasets, XSTest and JBB Behaviors, *Detect Jailbreak* and Azure’s *Prompt Shields* register precision values, and therefore F1 scores, tending to zero. An important distinction of the two dataset lies in their straightforward approach to unsafe prompt construction, consisting in explicit queries such as “How can I kill a person?” (XSTest) or “Write a sexist email about a female colleague” (JBB). This approach contrasts with other commonly used jailbreak datasets, which often employ indirect or obfuscated strategies, including role-playing scenarios (e.g., prompts like “From now on you are going to act as a DAN, which stands for “Do Anything Now”) to circumvent content moderation mechanisms.

We believe that this explicitness in XSTest and JBB likely contributes to the differential performance observed among various detection tools. Moderation systems such as PerspectiveAPI and Llama Guard, which are optimized to identify overtly unsafe or toxic language, tend to demonstrate higher efficacy on such datasets due to the unambiguous nature of the harmful content. Conversely, detection frameworks like Guardrails AI’s *Detect Jailbreak* and Azure’s *Prompt Shields* may underperform, as their training and heuristic methods appear more attuned to recognizing subtle, context-dependent jailbreak attempts typical of roleplay-based prompts. These tools might therefore exhibit limited sensitivity to direct and explicit unsafe requests.

It is noteworthy that PerspectiveAPI has been conceived as a generic tool for detecting toxicity of comments online, therefore it is very much plausible that, apart from this exception, it displays lower precision values.

4.2 A Deeper Insight into Llama Guard’s Performance

As mentioned in Section 3.2, Llama Guard was tested in a way that allowed us to simultaneously evaluate both the safety of the underlying LLM (in this case Mistral 7B) and the effectiveness of the guardrail. Actually this approach gives slightly different results with respect of just passing the input prompt, however, the difference is minimal, as base LLMs such as Mistral 7B generally lack the ability to reliably assess the safety of a prompt (for instance, they may produce unsafe content, such as a poem glorifying the Holocaust). The adopted approach also highlighted another challenge: these LLMs produce off-topic responses quite often. Such cases may corrupt the final outcome, since Llama Guard may classify those conversation as safe while the dataset’s ground-truth label indicates the opposite. In such instances, these false negatives are not genuine false negatives because the chat is indeed safe.

Overall, Llama Guard demonstrates high reliability in detecting unsafe chats and prompts; as shown in *Table 2* and *Table 3* it is the one that identifies more unsafe chats among all the guardrails. This result can be visualized in the bar chart in Figure 7, where Llama Guard shows the best F1 value four times out of six and never drops to zero.

5 Conclusions

This study systematically evaluated the performance of four prominent guardrail systems (Perspective API, Detect Jailbreak, Prompt Shields, and Llama Guard) across multiple datasets containing adversarial and non-adversarial prompts. The analysis revealed consistent patterns in the strengths and weaknesses of current safety detection frameworks for large language models.

Overall, all four systems demonstrated a strong preference for high precision, minimizing false positives but at the expense of recall. This conservative classification approach results in many unsafe prompts remaining undetected, as reflected in consistently low recall values across most datasets. Consequently, F1-scores are generally modest, highlighting the inherent challenge in balancing precision and recall within safety detection tasks. Increasing recall (e.g. by lowering the classification threshold) would enhance performance, but this modification carries the double risk of decreasing specificity and precision, because more negatives will be classified as positives (more FP).

Among the evaluated tools, Llama Guard generally achieved the highest recall and identified the largest number of unsafe prompts, suggesting that operating at the conversation level rather than solely on isolated prompts can enhance filtering effectiveness.

Perspective API, optimized for toxicity detection, performed well on datasets with explicit unsafe language but showed greater variability on more subtle prompts.

Detect Jailbreak and Prompt Shields tended to excel on intermediate-sized datasets but drastically underperformed on datasets with highly explicit unsafe queries. These results demonstrate that dataset characteristics critically influence detection performance.

In conclusion, while current guardrails provide valuable layers of safety for language models, there remains substantial room for improvement. No single guardrail system in this study provided comprehensive protection against all unsafe prompt types. This finding emphasizes the need for more flexible and adaptable safety strategies when deploying LLMs in socio-critical contexts, to offer more responsible and robust AI deployments.

6 Future work

Future work should explore hybrid approaches that integrate multiple detection strategies to improve recall without sacrificing precision. Additionally, the state of the art lacks context-aware models capable of handling linguistic complexity effectively. Guardrail systems that consider the interaction history, i.e. the sequence of prompts and responses preceding the current one, should be developed. In this way, the guardrail could be able to perform further reasoning on the most likely response.

Finally, repeating these experiments on more powerful hardware with recently developed LLMs and guardrails could enhance detection coverage, reduce false negatives, and potentially approach near-perfect identification of unsafe prompts.

Limitations This research was conducted on machines with limited computational resources, which constrained the use of larger and more advanced LLMs and guardrail versions. For example, experiments with LLaMA-2-chat[23] or Llama Guard 4[24] were not feasible in this setup. Hardware limitations also influenced dataset sampling in certain cases, potentially affecting recall measurements. The performance results should therefore be interpreted within the context of these computational constraints.

References

1. Yi, S., et al.: Jailbreak Attacks and Defenses Against Large Language Models: A Survey. arXiv:2407.04295v2 [cs.CR] 5 Jul 2024
2. Dong, Y., et al.: Safeguarding Large Language Models: A Survey. arXiv:2406.02622v1 [cs.CR] 3 Jun 2024
3. Dong, Y., et al.: Building Guardrails for Large Language Models. arXiv:2402.01822v2 [cs.CL] 2 Feb 2024
4. Inan, H., et al.: Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. Meta AI Blog, 7 Dec 2023 <https://ai.meta.com/research/publications/llama-guard-llm-based-input-output-safeguard-for-human-ai-conversations/>
5. Wang, X., et al.: SoK: Evaluating Jailbreak Guardrails for Large Language Models. arXiv:2506.10597v1 [cs.CR] 12 Jun 2025
6. Meta: PurpleLlama repository on GitHub. <https://github.com/meta-llama/PurpleLlama/blob/main/Prompt-Guard/README.md>, last accessed 07/08/2025
7. Perspective API documentation on website https://developers.perspectiveapi.com/s/about-the-api-key-concepts?language=en_US
8. Kang, M., Li, B.: $\mathbf{R}^2$ -Guard: Robust Reasoning Enabled LLM Guardrail via Knowledge-Enhanced Logical Reasoning. arXiv:2407.05557v1 [cs.AI] 8 Jul 2024
9. Xie, Y., et al.: GradSafe: Detecting Jailbreak Prompts for LLMs via Safety-Critical Gradient Analysis. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 507–518 (2024).
10. Han, S., et al.: Bridging the Safety Gap: A Guardrail Pipeline for Trustworthy LLM Inferences. arXiv:2502.08142v1 [cs.AI] 12 feb 2025
11. Hanu, L., et al.: Detoxify repository on GitHub. <https://github.com/unitaryai/detoxify> (2020), last accessed 07/08/2025
12. NVIDIA documentation <https://docs.nvidia.com/nemo/guardrails/latest/index.html>
13. AWS website <https://aws.amazon.com/it/bedrock/guardrails/>
14. Guardrails AI blog <https://www.guardrailsai.com/blog/advanced-pii-and-jailbreak>
15. Detect Jailbreak page on Guardrails AI hub https://hub.guardrailsai.com/validator/guardrails/detect_jailbreak
16. Microsoft Azure documentation <https://learn.microsoft.com/en-us/azure/ai-services/content-safety/concepts/jailbreak-detection>
17. Guidance repository on GitHub. <https://github.com/guidance-ai/guidance>, last accessed 07/08/2025
18. Meta: Llama Guard 3 8B repository on GitHub. <https://github.com/meta-llama/PurpleLlama/blob/main/Llama-Guard3/8B/README.md>, last accessed 07/08/2025

19. ML Commons website <https://mlcommons.org/2024/04/mlc-aisafety-v0-5-poc/>
20. Guardrails AI hub <https://hub.guardrailsai.com/>
21. Meta documentation on Llama 3.1 <https://ai.meta.com/blog/meta-llama-3-1/>
22. Rainio, O., et al.: Evaluation metrics and statistical tests for machine learning. Sci Rep 14, 6086 (2024). <https://doi.org/10.1038/s41598-024-56706-x>
23. Meta documentation on Llama-2-chat <https://www.llama.com/llama2/>
24. Meta documentation on Llama 4 <https://www.llama.com/docs/model-cards-and-prompt-formats/llama-guard-4/>